

Chapter 8

Tuples

This chapter presents one more built-in type, the tuple, and then shows how lists, and tuples work together. I also present a useful feature for variable-length argument lists, the gather and scatter operators.

One note: there is no consensus on how to pronounce “tuple”. Some people say “tuh-ple”, which rhymes with “supple”. But in the context of programming, most people say “too-ple”, which rhymes with “quadruple”.

8.1 Tuples are immutable

A tuple is a sequence of values. The values can be any type, and they are indexed by integers, so in that respect tuples are a lot like lists. The important difference is that tuples are immutable.

Syntactically, a tuple is a comma-separated list of values:

```
t = 'a', 'b', 'c', 'd', 'e'
```

Although it is not necessary, it is common to enclose tuples in parentheses:

```
t = ('a', 'b', 'c', 'd', 'e')
```

To create a tuple with a single element, you have to include a final comma:

```
t1 = 'a',  
type(t1)
```

```
<class 'tuple'>
```

A value in parentheses is not a tuple:

```
t2 = ('a')  
type(t2)
```

```
<class 'str'>
```

Another way to create a tuple is the built-in function `tuple`. With no argument, it creates an empty tuple:

```
t = tuple()
t
```

```
()
```

If the argument is a sequence (string, list or tuple), the result is a tuple with the elements of the sequence:

```
t = tuple('lupins')
t
```

```
('l', 'u', 'p', 'i', 'n', 's')
```

Because `tuple` is the name of a built-in function, you should avoid using it as a variable name.

Most list operators also work on tuples. The bracket operator indexes an element:

```
t = ('a', 'b', 'c', 'd', 'e')
t[0]
```

```
'a'
```

And the slice operator selects a range of elements.

```
t[1:3]
```

```
('b', 'c')
```

But if you try to modify one of the elements of the tuple, you get an error:

```
t[0] = 'A'
```

```
TypeError: object doesn't support item assignment
```

Because tuples are immutable, you can't modify the elements. But you can replace one tuple with another:

```
t = ('A',) + t[1:]
t
```

```
('A', 'b', 'c', 'd', 'e')
```

This statement makes a new tuple and then makes `t` refer to it.

The relational operators work with tuples and other sequences; Python starts by comparing the first element from each sequence. If they are equal, it goes on to the next elements, and so on, until it finds elements that differ. Subsequent elements are not considered (even if they are really big).

```
(0, 1, 2) < (0, 3, 4)
```

```
True
```

```
(0, 1, 2000000) < (0, 3, 4)
```

```
True
```

8.2 Tuple assignment

It is often useful to swap the values of two variables. With conventional assignments, you have to use a temporary variable. For example, to swap `a` and `b`:

```
temp = a
a = b
b = temp
```

This solution is cumbersome; **tuple assignment** is more elegant:

```
a, b = b, a
```

The left side is a tuple of variables; the right side is a tuple of expressions. Each value is assigned to its respective variable. All the expressions on the right side are evaluated before any of the assignments.

The number of variables on the left and the number of values on the right have to be the same:

```
a, b = 1, 2, 3
```

```
ValueError: too many values to unpack
```

More generally, the right side can be any kind of sequence (string, list or tuple). For example, to split an email address into a user name and a domain, you could write:

```
addr = 'monty@python.org'
uname, domain = addr.split('@')
```

The return value from `split` is a list with two elements; the first element is assigned to `uname`, the second to `domain`.

```
uname
```

```
'monty'
```

```
domain
```

```
'python.org'
```

8.3 Tuples as return values

Strictly speaking, a function can only return one value, but if the value is a tuple, the effect is the same as returning multiple values. For example, if you want to divide two integers and compute the quotient and remainder, it is inefficient to compute `x//y` and then `x%y`. It is better to compute them both at the same time.

The built-in function `divmod` takes two arguments and returns a tuple of two values, the quotient and remainder. You can store the result as a tuple:

```
t = divmod(7, 3)
t
```

```
(2, 1)
```

Or use tuple assignment to store the elements separately:

```
quot, rem = divmod(7, 3)
quot
```

```
2
```

```
rem
```

```
1
```

Here is an example of a function that returns a tuple:

```
def min_max(t):
    return min(t), max(t)
```

`max` and `min` are built-in functions that find the largest and smallest elements of a sequence. `min_max` computes both and returns a tuple of two values.

8.4 Variable-length argument tuples

Functions can take a variable number of arguments. A parameter name that begins with `*` **gathers** arguments into a tuple. For example, `printall` takes any number of arguments and prints them:

```
def printall(*args):
    print(args)
```

The gather parameter can have any name you like, but `args` is conventional. Here's how the function works:

```
printall(1, 2.0, '3')
```

```
(1, 2.0, '3')
```

The complement of gather is **scatter**. If you have a sequence of values and you want to pass it to a function as multiple arguments, you can use the `*` operator. For example, `divmod` takes exactly two arguments; it doesn't work with a tuple:

```
t = (7, 3)
divmod(t)
```

```
TypeError: divmod expected 2 arguments, got 1
```

But if you scatter the tuple, it works:

```
divmod(*t)
```

```
(2, 1)
```

Many of the built-in functions use variable-length argument tuples. For example, `max` and `min` can take any number of arguments:

```
max(1, 2, 3)
```

```
3
```

But `sum` does not.

```
sum(1, 2, 3)
```

```
TypeError: sum expected at most 2 arguments, got 3
```

As an exercise, write a function called `sum_all` that takes any number of arguments and returns their sum.

8.5 Lists and tuples

`zip` is a built-in function that takes two or more sequences and interleaves them. The name of the function refers to a zipper, which interleaves two rows of teeth.

This example zips a string and a list:

```
s = 'abc'
t = [0, 1, 2]
zip(s, t)
```

```
<zip object at 0x7f7d0a9e7c48>
```

The result is a **zip object** that knows how to iterate through the pairs. The most common use of `zip` is in a `for` loop:

```
for pair in zip(s, t):
    print(pair)
```

```
('a', 0)
('b', 1)
('c', 2)
```

A `zip` object is a kind of **iterator**, which is any object that iterates through a sequence. Iterators are similar to lists in some ways, but unlike lists, you can't use an index to select an element from an iterator.

If you want to use list operators and methods, you can use a `zip` object to make a list:

```
list(zip(s, t))
```

```
[('a', 0), ('b', 1), ('c', 2)]
```

The result is a list of tuples; in this example, each tuple contains a character from the string and the corresponding element from the list.

If the sequences are not the same length, the result has the length of the shorter one.

```
list(zip('Anne', 'Elk'))
```

```
[('A', 'E'), ('n', 'l'), ('n', 'k')]
```

You can use tuple assignment in a `for` loop to traverse a list of tuples:

```
t = [('a', 0), ('b', 1), ('c', 2)]
for letter, number in t:
    print(number, letter)
```

Each time through the loop, Python selects the next tuple in the list and assigns the elements to `letter` and `number`. The output of this loop is:

```
0 a
1 b
2 c
```

If you combine `zip`, `for` and tuple assignment, you get a useful idiom for traversing two (or more) sequences at the same time. For example, `has_match` takes two sequences, `t1` and `t2`, and returns `True` if there is an index `i` such that `t1[i] == t2[i]`:

```
def has_match(t1, t2):
    for x, y in zip(t1, t2):
        if x == y:
            return True
    return False
```

If you need to traverse the elements of a sequence and their indices, you can use the built-in function `enumerate`:

```
for index, element in enumerate('abc'):
    print(index, element)
```

The result from `enumerate` is an `enumerate` object, which iterates a sequence of pairs; each pair contains an index (starting from 0) and an element from the given sequence. In this example, the output is

```
0 a
1 b
2 c
```

Again.

8.6 Sequences of sequences

I have focused on lists of tuples, but almost all of the examples in this chapter also work with lists of lists, tuples of tuples, and tuples of lists. To avoid enumerating the possible combinations, it is sometimes easier to talk about sequences of sequences.

In many contexts, the different kinds of sequences (strings, lists and tuples) can be used interchangeably. So how should you choose one over the others?

To start with the obvious, strings are more limited than other sequences because the elements have to be characters. They are also immutable. If you need the ability to change the characters in a string (as opposed to creating a new string), you might want to use a list of characters instead.

Lists are more common than tuples, mostly because they are mutable. But there are a few cases where you might prefer tuples:

1. In some contexts, like a `return` statement, it is syntactically simpler to create a tuple than a list.
2. If you want to use a sequence as a dictionary key, you have to use an immutable type like a tuple or string.
3. If you are passing a sequence as an argument to a function, using tuples reduces the potential for unexpected behavior due to aliasing.

Because tuples are immutable, they don't provide methods like `sort` and `reverse`, which modify existing lists. But Python provides the built-in function `sorted`, which takes any sequence and returns a new list with the same elements in sorted order, and `reversed`, which takes a sequence and returns an iterator that traverses the list in reverse order.

8.7 Debugging

Lists and tuples are examples of **data structures**; in this chapter we are starting to see compound data structures, like lists of tuples. Compound data structures are useful, but they are prone to what I call **shape errors**; that is, errors caused when a data structure has the wrong type, size, or structure. For example, if you are expecting a list with one integer and I give you a plain old integer (not in a list), it won't work.

To help debug these kinds of errors, I have written a module called `structshape` that provides a function, also called `structshape`, that takes any kind of data structure as an argument and returns a string that summarizes its shape. You can download it from <http://thinkpython2.com/code/structshape.py>

Here's the result for a simple list:

```
from structshape import structshape
t = [1, 2, 3]
structshape(t)
```

```
'list of 3 int'
```

A fancier program might write "list of 3 ints", but it was easier not to deal with plurals. Here's a list of lists:

```
t2 = [[1,2], [3,4], [5,6]]
structshape(t2)
```

```
'list of 3 list of 2 int'
```

If the elements of the list are not the same type, `structshape` groups them, in order, by type:

```
t3 = [1, 2, 3, 4.0, '5', '6', [7], [8], 9]
structshape(t3)
```

```
'list of (3 int, float, 2 str, 2 list of int, int)'
```

Here's a list of tuples:

```
s = 'abc'
lt = list(zip(t, s))
structshape(lt)
```

```
'list of 3 tuple of (int, str)'
```

If you are having trouble keeping track of your data structures, `structshape` can help.

8.8 Glossary

tuple: An immutable sequence of elements.

tuple assignment: An assignment with a sequence on the right side and a tuple of variables on the left. The right side is evaluated and then its elements are assigned to the variables on the left.

gather: An operation that collects multiple arguments into a tuple.

- scatter:** An operation that makes a sequence behave like multiple arguments.
- zip object:** The result of calling a built-in function `zip`; an object that iterates through a sequence of tuples.
- iterator:** An object that can iterate through a sequence, but which does not provide list operators and methods.
- data structure:** A collection of related values, often organized in lists, dictionaries, tuples, etc.
- shape error:** An error caused because a value has the wrong shape; that is, the wrong type or size.

8.9 Exercises

Exercise 1:

Write a function called `most_frequent` that takes a string and prints the letters in decreasing order of frequency. Find text samples from several different languages and see how letter frequency varies between languages. Compare your results with the tables at http://en.wikipedia.org/wiki/Letter_frequencies. Solution: http://thinkpython2.com/code/most_frequent.py.

Exercise 2:

More anagrams!

1. Write a program that reads a word list from a file and prints all the sets of words that are anagrams.

Here is an example of what the output might look like:

```
['deltas', 'desalt', 'lasted', 'salted', 'slated', 'staled']
['retainers', 'ternaries']
['generating', 'greatening']
['resmelts', 'smelters', 'termless']
```

Hint: you might want to build a dictionary that maps from a collection of letters to a list of words that can be spelled with those letters. The question is, how can you represent the collection of letters in a way that can be used as a key?

2. Modify the previous program so that it prints the longest list of anagrams first, followed by the second longest, and so on.
3. In Scrabble a “bingo” is when you play all seven tiles in your rack, along with a letter on the board, to form an eight-letter word. What collection of 8 letters forms the most possible bingos?

Solution: http://thinkpython2.com/code/anagram_sets.py.

Exercise 3:

Two words form a “metathesis pair” if you can transform one into the other by swapping two letters; for example, “converse” and “conserve”. Write a program that finds all of the metathesis pairs in the dictionary. Hint: don’t test all pairs of words, and don’t test all possible swaps. Solution: <http://thinkpython2.com/code/metathesis.py>. Credit: This exercise is inspired by an example at <http://puzzlers.org>.

Exercise 4:

Here’s another Car Talk Puzzler (<http://www.cartalk.com/content/puzzlers>):

What is the longest English word, that remains a valid English word, as you remove its letters one at a time?

Now, letters can be removed from either end, or the middle, but you can’t rearrange any of the letters. Every time you drop a letter, you wind up with another English word. If you do that, you’re eventually going to wind up with one letter and that too is going to be an English word—one that’s found in the dictionary. I want to know what’s the longest word and how many letters does it have?

I’m going to give you a little modest example: Sprite. Ok? You start off with sprite, you take a letter off, one from the interior of the word, take the r away, and we’re left with the word spite, then we take the e off the end, we’re left with spit, we take the s off, we’re left with pit, it, and I.

Write a program to find all words that can be reduced in this way, and then find the longest one.

This exercise is a little more challenging than most, so here are some suggestions:

1. You might want to write a function that takes a word and computes a list of all the words that can be formed by removing one letter. These are the “children” of the word.
2. Recursively, a word is reducible if any of its children are reducible. As a base case, you can consider the empty string reducible.
3. The wordlist I provided, `words.txt`, doesn’t contain single letter words. So you might want to add “I”, “a”, and the empty string.
4. To improve the performance of your program, you might want to memoize the words that are known to be reducible.

Solution: <http://thinkpython2.com/code/reducible.py>.

